PROJETO: Startpoint



PUC
CAMPINAS
PONTIFÍCIA UNIVERSIDADE CATÓLICA

Curso: Ciência de Dados e IA

Disciplina: PI: Programação Web **Código:** 2181161

Turma: 102

Professor: Fernando Luiz de Almeida Silveira

Nome do Grupo: Startpoint

Caio Furlan – RA: 25005477

Conrado Marques – RA: 25009450

Otavio Astolfo Torato – RA: 25021678

Igor Novais Ribeiro Santos – RA: 25021610

Bruno Teixeira Souto Maior – RA: 25008760

Campinas, 2º semestre de 2026

1. Ideação do Projeto

1.1 Problema

O Brasil tem uma rede grande de cursos gratuitos de qualificação, oferecidos por instituições como SENAI, SENAC, Fundação Bradesco, SEBRAE e Escola Virtual de Governo. Essa oferta, porém, é espalhada: cada instituição divulga os cursos no próprio site, com calendários e critérios diferentes.

Para o jovem que procura o primeiro emprego, isso gera três dificuldades. Ele não sabe o que existe, porque descobre os cursos por acaso. Não sabe o que a vaga exige, porque os anúncios de Jovem Aprendiz trazem requisitos genéricos. E não sabe em que ordem estudar, porque ninguém o orienta sobre a sequência adequada para a área que deseja. O problema não é a falta de oportunidades, mas a falta de um caminho claro entre a situação atual do jovem e a vaga que ele quer. As consequências são tempo perdido com cursos que não ajudam, desistência por falta de progresso visível e mais tempo fora do mercado formal.

A relevância aparece nos indicadores oficiais: no 2º trimestre de 2025, a taxa de desocupação dos jovens de 18 a 24 anos foi de 12%, mais que o dobro da média nacional, de 5,8% (PNAD Contínua/IBGE).

1.2 Público-alvo e contexto

O público principal são jovens de 16 a 24 anos em busca do primeiro emprego formal, cursando ou tendo concluído o ensino médio, em sua maioria moradores de regiões periféricas de grandes cidades. É um público que acessa a internet principalmente pelo celular, muitas vezes com plano de dados limitado, e que não tem rede de contatos profissionais nem orientação familiar sobre carreira. O uso acontece em casa, na escola ou no transporte, em sessões curtas.

Como público secundário, o Startpoint atende organizações do terceiro setor que trabalham com empregabilidade juvenil e que podem usar o aplicativo para orientar os jovens que acompanham.

Abrangência: nesta versão (MVP), o Startpoint atende **apenas a cidade de São Paulo/SP**. Os cursos e as vagas cadastrados são de São Paulo, e a etapa 3 do diagnóstico mostra só essa cidade, com o aviso “Por enquanto o Startpoint atende a cidade de São Paulo”. Outras cidades da região metropolitana ficam para as próximas versões (seção 8.2).

1.3 Setor da sociedade e ODS

O projeto está no **Terceiro Setor**. O aplicativo não tem finalidade comercial: não cobra do usuário, não intermedeia contratações e não substitui o serviço público de emprego. Ele apoia o trabalho que organizações sem fins lucrativos já fazem, ampliando o alcance delas.

As ODS relacionadas são:

- **ODS 4 – Educação de Qualidade** (meta 4.4): aumentar o número de jovens com competências para o emprego, conectando o jovem aos cursos gratuitos em uma sequência com propósito.
- **ODS 8 – Trabalho Decente e Crescimento Econômico** (meta 8.6): reduzir a proporção de jovens sem emprego, educação ou formação, encurtando o caminho até a primeira vaga formal.
- **ODS 10 – Redução das Desigualdades** (complementar): quem tem uma boa rede de contatos enxerga oportunidades que os outros não veem; organizar essa informação reduz uma desigualdade de partida.

1.4 Tipo de impacto

O impacto principal é **social**: o jovem passa a ter acesso organizado à informação sobre cursos e vagas, sem depender do acaso. Há também impacto **econômico**, porque o primeiro emprego formal significa renda, registro em carteira e início do histórico previdenciário, e porque as empresas recebem candidatos mais preparados para as vagas de aprendiz.

1.5 Solução proposta

O Startpoint transforma um objetivo profissional em um caminho concreto de estudo. O jovem informa a área que deseja e o que já sabe; o sistema compara esse perfil com os requisitos das vagas de aprendiz daquela área e devolve uma trilha ordenada de cursos gratuitos que cobre o que falta. Depois, acompanha o progresso e mostra o quanto o perfil do jovem combina com cada vaga. A diferença para um site de busca comum é que o aplicativo não devolve uma lista de tudo o que existe, e sim a sequência que aquele usuário precisa cumprir.

1.6 Principais funcionalidades

- **Perfil e diagnóstico inicial:** cadastro e quatro perguntas (escolaridade, o que já aprendeu, cidade e área de interesse).
- **Trilha personalizada:** sequência de cursos montada a partir do que as vagas da área pedem e o jovem ainda não tem.
- **Catálogo de cursos gratuitos:** lista dos cursos por área, com a indicação “Na trilha”.
- **Mural de vagas de aprendiz:** percentual de aderência de cada vaga e o bloco “O que falta”, com o nome de um curso que ensina cada requisito e atalho para o catálogo.
- **Acompanhamento de progresso:** marcação de cursos concluídos, barra de evolução e recálculo da aderência às vagas.
- **Checklist de contratação:** documentos exigidos para a contratação de aprendiz e dicas de currículo e entrevista (ficou para depois do MVP, por causa do prazo).

1.7 Viabilidade técnica

O projeto é viável no semestre porque o escopo do MVP foi limitado: uma cidade (São Paulo/SP), cinco áreas profissionais com dados e uma base de cursos e vagas montada pelo grupo. A parte mais complexa, o motor da trilha, foi resolvida com regras simples de consulta e ordenação no banco de dados, sem inteligência artificial. HTML, CSS, JavaScript e MySQL são conteúdos das disciplinas de Programação Web e Bancos de Dados Relacionais, e o back-end usa Node.js, que roda o mesmo JavaScript visto em aula. A seção 5 mostra que, nesta 3ª entrega, o sistema já funciona de ponta a ponta.

1.8 Valor entregue

O valor central é a **inclusão social**: uma informação que hoje chega de forma desigual passa a estar organizada para qualquer jovem, com ou sem rede de contatos. Como efeitos secundários, o jovem estuda com direção, o que tende a reduzir a desistência dos cursos gratuitos; chega mais perto do primeiro emprego formal; e as organizações de empregabilidade conseguem orientar mais jovens sem aumentar a equipe.

2. Protótipo da Solução

O protótipo foi feito no Figma Make como aplicação **web responsiva**, pensada primeiro para a tela do celular, com modo claro e escuro e a identidade visual laranja, preta e branca do Startpoint. Ele serviu de base para as telas desenvolvidas em HTML, CSS e JavaScript (seção 5). O protótipo completo pode ser acessado em figma.com/make/q4EgNLn6MMgyzlzpJ5AGTR/Prototipo.

2.1 Telas principais

- **Acesso e Criar conta:** login por e-mail e senha, recuperação de senha e cadastro com confirmação de senha.
- **Diagnóstico inicial:** quatro etapas com barra de progresso (“Etapa X de 4”): estudos, o que já aprendeu, cidade e área de interesse. A última etapa tem o botão “Gerar minha trilha”.
- **Início:** saudação, percentual da trilha, aderência média às vagas, próxima ação recomendada e atalho para o checklist.
- **Minha trilha:** cursos em ordem sugerida, com instituição, duração, modalidade e situação, e os botões “Marcar como concluído” e “Adicionar outro curso”.
- **Cursos:** catálogo com busca, filtro e a indicação “Na trilha”.
- **Vagas:** vagas filtradas pela cidade e pela área do usuário, com aderência, modalidade, bolsa e o bloco “O que falta”.
- **Checklist de contratação e Perfil:** documentos para a contratação, dicas, dados do diagnóstico e os botões “Editar diagnóstico” e “Sair da conta”.

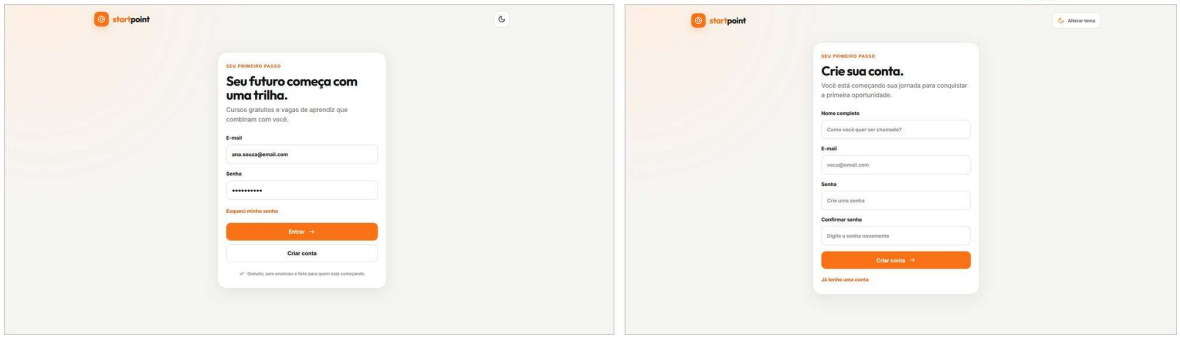


Figura 1 – Protótipo: telas de acesso e de criação de conta.

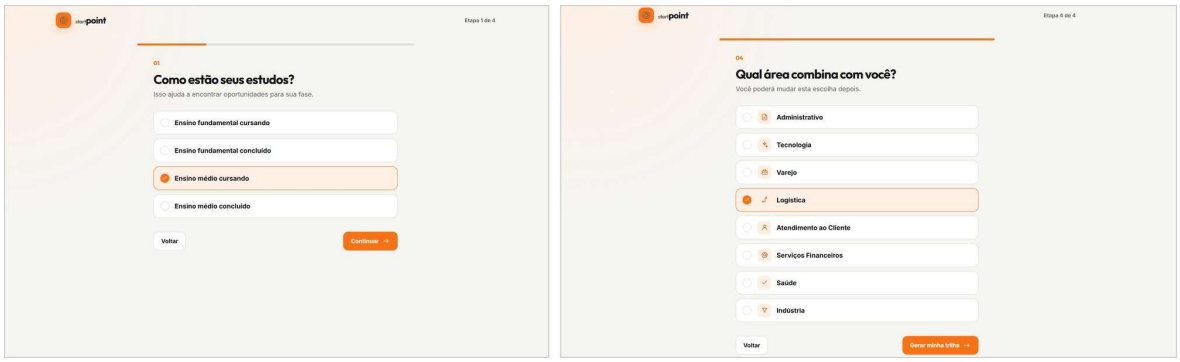


Figura 2 – Protótipo: primeira e última etapas do diagnóstico.

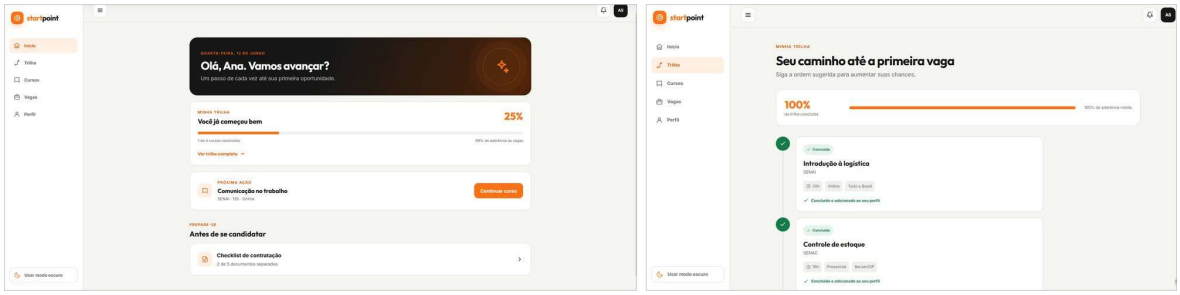


Figura 3 – Protótipo: telas Início e Minha trilha.

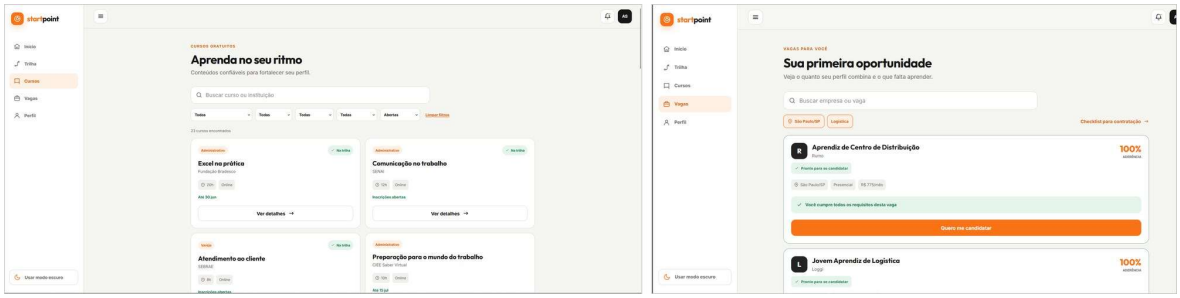


Figura 4 – Protótipo: telas Cursos e Vagas.

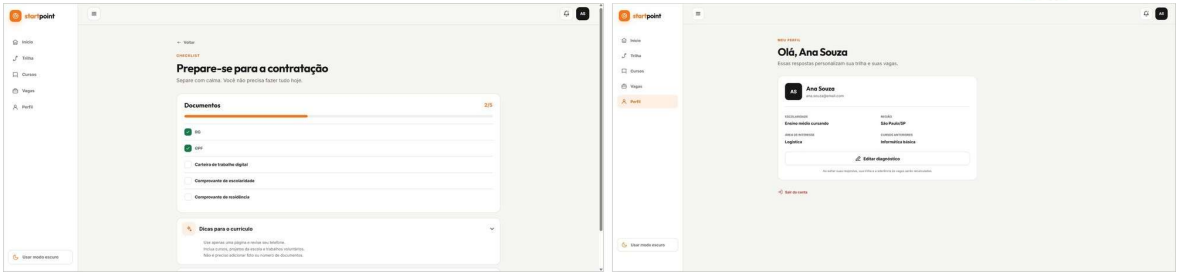


Figura 5 – Protótipo: telas Checklist de contratação e Perfil.

2.2 Fluxo de navegação e interação

O fluxo começa na tela de acesso. Quem ainda não tem conta passa por Criar conta e volta para entrar. No primeiro acesso, o jovem responde ao diagnóstico, o sistema gera a trilha e ele chega ao Início. A partir daí, o menu (Início, Trilha, Cursos, Vagas e Perfil), que fica na lateral no computador e no topo no celular, leva às demais telas. O jovem segue a trilha, marca os cursos concluídos e acompanha o progresso. Nas Vagas, vê o quanto seu perfil combina com cada oportunidade e o que falta aprender. Se a situação mudar, edita o diagnóstico pelo Perfil. No protótipo, o jovem também consulta o checklist de contratação antes de se candidatar.

2.3 Coerência com a proposta

As três dificuldades do problema aparecem resolvidas nas telas: quem não sabe o que existe encontra os cursos reunidos no catálogo; quem não sabe o que a vaga exige vê a aderência e o bloco “O que falta”; e quem não sabe em que ordem estudar recebe a trilha. As seis funcionalidades previstas estão representadas, e o layout com textos curtos e botões grandes respeita o público, que usa principalmente o celular.

Na implementação, o grupo simplificou alguns pontos para caber no prazo do semestre: o checklist de contratação, a busca das telas Cursos e Vagas e os filtros da tela Cursos ficaram fora do MVP; a escolha de cidade ficou só com São Paulo/SP; a janela de candidatura virou uma confirmação simples antes de abrir o site da vaga; e o botão “Ver vagas em outras cidades” saiu, porque o MVP tem uma única cidade.

3. Planejamento do Projeto

3.1 Product Backlog e priorização

O backlog foi definido na 2ª entrega a partir das seis funcionalidades da proposta, com a prioridade de cada item. A Tabela 1 mostra o backlog e a situação de cada item nesta 3ª entrega.

Tabela 1 – Product Backlog e situação na 3ª entrega

Item	Funcionalidade	Prioridade	Sprint prevista	Situação
1	Cadastro e login (e-mail e senha)	Alta	Sprint 1	Concluído (telas na Sprint 1, API na Sprint 2)
2	Perfil e diagnóstico do jovem	Alta	Sprint 1	Concluído (telas na Sprint 1, API na Sprint 2)
3	Catálogo de cursos com filtros	Alta	Sprint 2	Parcial: lista por área, sem busca e sem filtros
4	Geração da trilha personalizada	Alta	Sprint 2	Concluído
5	Mural de vagas com % de aderência	Média	Sprint 3	Adiantado para a Sprint 2 e concluído
6	Acompanhamento de progresso	Média	Sprint 3	Adiantado para a Sprint 2 e concluído
7	Checklist de preparação	Baixa	Sprint 3	Retirado do MVP por causa do prazo
8	PWA (manifest.json e service worker)	Baixa	Sprint 3	A fazer

3.2 Organização com Scrum

O trabalho foi dividido em três sprints, uma para cada entrega que faltava depois da proposta. Cada sprint começa com uma reunião de planejamento, em que o grupo escolhe os itens do backlog, e termina com uma revisão do que ficou pronto e uma retrospectiva rápida. Durante a sprint, o grupo faz um alinhamento curto duas vezes por semana, presencial ou pelo grupo de mensagens.

- **Sprint 1 (2ª entrega):** protótipo, planejamento, quadro Kanban, modelagem do banco, estrutura do repositório e primeiras telas em HTML.
- **Sprint 2 (3ª entrega):** sistema integrado. Estava prevista para os itens 3 e 4, mas, como a 3ª entrega pede front-end, back-end e banco funcionando juntos, o grupo adiantou os itens 5 e 6, que usam as mesmas tabelas e rotas. O item 7 (checklist) foi retirado do MVP para o grupo concentrar o tempo no sistema principal. Foi também nesta sprint que o cadastro, o login e o diagnóstico (itens 1 e 2), cujas telas vinham da Sprint 1, passaram a funcionar com a API.
- **Sprint 3 (4ª entrega):** PWA, carga de cursos e vagas reais revisada com o parceiro da comunidade, testes com usuários e ajustes finais de layout.

3.3 Divisão das responsabilidades

Os papéis do Scrum definidos na 2ª entrega foram mantidos. Nesta sprint, o Otavio passou a cuidar também do back-end, e cada parte do código ganhou um responsável por mantê-la no repositório:

- **Caio Furlan – Product Owner:** prioriza o backlog e confere se o que foi entregue atende à proposta.
- **Bruno Teixeira Souto Maior – Scrum Master:** organiza as reuniões de planejamento, revisão e retrospectiva e acompanha o ritmo da sprint.
- **Igor Novais Ribeiro Santos – Banco de dados e repositório:** scripts de criação, dados de exemplo e consultas (pasta database).
- **Conrado Marques – Front-end:** telas em HTML, CSS e JavaScript e a comunicação com a API (pasta frontend).
- **Otavio Astolfo Torato – Back-end, Kanban e documentação:** API em Node.js (pasta backend), README, quadro Kanban e documentos das entregas.

3.4 Quadro Kanban

O grupo usa um quadro no Trello, chamado startpoint, com as colunas **A Fazer**, **Em Andamento** e **Concluído**. Cada tarefa é um cartão com título, descrição e prazo. No início da sprint, as tarefas escolhidas entram em A Fazer; quem começa uma tarefa move o cartão para Em Andamento e, ao terminar, para Concluído. Nos alinhamentos, o responsável pelo Kanban aponta os cartões parados ou com prazo próximo. A Tabela 2 resume as tarefas da Sprint 2, e a figura logo depois dela mostra o quadro no fim da sprint.

Tabela 2 – Tarefas da Sprint 2

Tarefa	Responsável	Situação
Ajustar o banco: tabela usuario_competencia, área e link na vaga, índices	Igor	Concluído
Script de dados de exemplo e arquivo de consultas	Igor	Concluído
Estilizar as telas e criar Vagas e Perfil	Conrado	Concluído
Ligar as telas à API com JavaScript (fetch)	Conrado	Concluído
API em Node.js com Express: login com JWT, trilha, cursos e vagas	Otavio	Concluído
Teste automático da API e README com instruções	Otavio	Concluído
Planejamento e revisão da sprint	Caio e Bruno	Concluído
Documento e apresentação da 3ª entrega	Todos	Em andamento
Carga de cursos e vagas reais com o parceiro	Todos	A fazer (Sprint 3)
PWA (manifest.json e service worker)	Conrado	A fazer (Sprint 3)

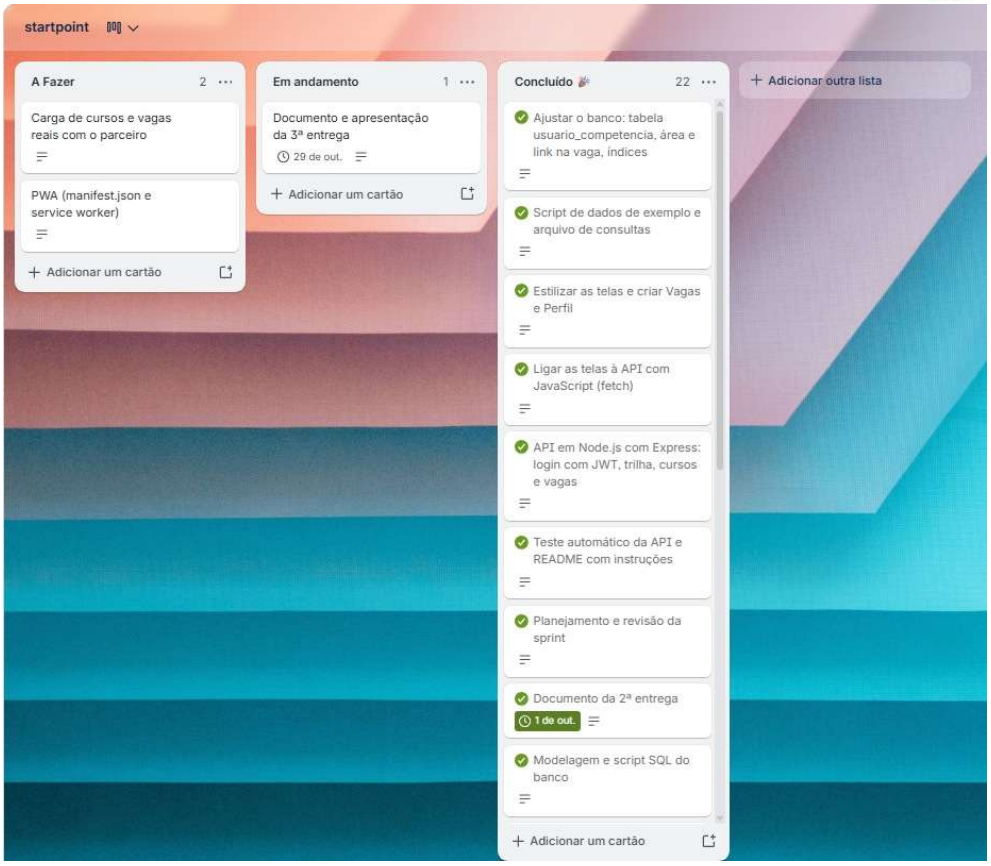


Figura 6 – Quadro Kanban no Trello ao final da Sprint 2.

3.5 Como o planejamento garante o MVP

O Scrum define o que entra em cada etapa e o Kanban mostra, a cada momento, como esse trabalho está andando. Com isso, o grupo percebeu cedo que dava para adiantar os itens 5 e 6 na Sprint 2 e que era melhor deixar o checklist, a busca e os filtros de cursos fora do MVP. Para a Sprint 3 ficaram o PWA, o que depende de terceiros (dados reais com o parceiro e testes com usuários) e os ajustes apontados nos testes. A divisão por pastas (banco, front-end e back-end) também permitiu que cada integrante trabalhasse em paralelo sem atrapalhar o outro.

4. Levantamento de Requisitos

Os requisitos foram levantados a partir da proposta da 1ª entrega e do protótipo. A coluna “Situação” indica o que já está implementado nesta entrega.

4.1 Requisitos funcionais

Tabela 3 – Requisitos funcionais

Código	Descrição	Situação
RF01	Cadastrar usuário com nome, e-mail, senha e confirmação de senha.	Implementado
RF02	Autenticar o usuário por e-mail e senha e manter a sessão por token.	Implementado

Código	Descrição	Situação
RF03	Recuperar a senha pelo e-mail cadastrado.	Parcial: a tela existe, o envio do e-mail não
RF04	Registrar o diagnóstico em quatro etapas e permitir editá-lo pelo Perfil.	Implementado
RF05	Gerar a trilha de cursos a partir do diagnóstico e das vagas da área.	Implementado
RF06	Listar os cursos ativos, organizados por área, indicando os que já estão na trilha.	Implementado
RF07	Adicionar e remover cursos da trilha.	Implementado
RF08	Marcar e desmarcar cursos como concluídos e mostrar o percentual da trilha.	Implementado
RF09	Listar vagas com filtros de cidade e área, mostrando a aderência e o que falta.	Implementado
RF10	Indicar, para cada requisito que falta, um curso que o ensina.	Implementado
RF11	Encaminhar o jovem ao site da vaga, com uma confirmação antes.	Implementado
RF12	Mostrar o perfil do usuário e permitir sair da conta.	Implementado
RF13	Alternar entre tema claro e escuro e lembrar a escolha.	Implementado
RF14	Permitir instalar o sistema na tela inicial do celular (PWA).	Sprint 3

4.2 Requisitos não funcionais

Tabela 4 – Requisitos não funcionais

Código	Categoria	Descrição
RNF01	Segurança	Senhas guardadas apenas como hash bcrypt; rotas protegidas por token JWT com validade de 8 horas; consultas SQL com parâmetros (?), sem montar SQL com o texto digitado.
RNF02	Segurança	Cada usuário só consulta e altera a própria trilha: o id do usuário vem do token, e não da tela.
RNF03	Usabilidade	Textos curtos, botões grandes e mensagens de erro em português, explicando o que fazer.
RNF04	Responsividade	As telas devem funcionar no celular (a partir de 360 px de largura) e no computador, sem rolagem lateral da página.
RNF05	Compatibilidade	Funcionar nos navegadores atuais (Chrome, Edge e Firefox), sem instalar nada.
RNF06	Desempenho	Índices nas colunas usadas nos filtros (área do curso, área e cidade da vaga).
RNF07	Manutenibilidade	Back-end separado em rotas, controllers e models; no front-end, um script para cada tela (as quatro etapas do diagnóstico usam o mesmo) e arquivos compartilhados para a API e o tema.
RNF08	Privacidade	Coletar só os dados necessários para a trilha (nome, e-mail, escolaridade, cidade, área e competências).

4.3 Regras de negócio

Tabela 5 – Regras de negócio

Código	Regra
RN01	O e-mail é único: não é possível criar duas contas com o mesmo e-mail.
RN02	O nome precisa ter pelo menos 3 letras e a senha pelo menos 6 caracteres.
RN03	As competências do jovem são as marcadas no diagnóstico mais as ensinadas pelos cursos que ele concluiu.
RN04	A trilha considera apenas as competências pedidas pelas vagas ativas da área escolhida que o jovem ainda não tem.
RN05	Na escolha dos cursos, os da área do jovem vêm primeiro; depois, os que cobrem mais requisitos e os mais curtos. Um curso que não ensina nada novo não entra na trilha.
RN06	Se a área ainda não tem vagas cadastradas, a trilha recebe até 5 cursos da própria área, do mais curto para o mais longo.
RN07	Ao refazer o diagnóstico, os cursos concluídos continuam na trilha e os pendentes são recalculados.
RN08	Um curso não pode aparecer duas vezes na mesma trilha.
RN09	A aderência a uma vaga é a proporção dos requisitos da vaga (obrigatórios e desejáveis) que o jovem já tem.
RN10	Cursos e vagas inativos não aparecem no catálogo e não entram em trilhas novas, mas não são apagados, para preservar o histórico.

A aderência (RN09) e o progresso da trilha são calculados assim:

$$\text{Aderência (\%)} = \frac{\text{requisitos da vaga que o jovem já tem}}{\text{total de requisitos da vaga}} \times 100$$

$$\text{Progresso da trilha (\%)} = \frac{\text{cursos concluídos}}{\text{total de cursos da trilha}} \times 100$$

4.4 Restrições do sistema

- O MVP atende apenas a cidade de São Paulo/SP.
- Há cursos e vagas cadastrados para cinco áreas: Administrativo, Tecnologia, Varejo, Logística e Saúde. As áreas Atendimento ao Cliente, Serviços Financeiros e Indústria aparecem no diagnóstico, mas ainda não têm dados.
- A base de cursos e vagas é mantida por scripts SQL executados pelo grupo; não há integração automática com os sites das instituições.
- O checklist de contratação, a busca por texto nas telas Cursos e Vagas e os filtros da tela Cursos ficaram fora do MVP.
- A candidatura é feita no site da empresa ou do programa de aprendizagem, fora do Startpoint.
- O sistema é web: precisa de navegador, internet e do servidor da API ligado.

5. Desenvolvimento do Projeto

Nesta sprint o Startpoint passou de telas estáticas para um sistema integrado: as telas em HTML, CSS e JavaScript conversam com uma API em Node.js, que grava e consulta os dados no MySQL. O código está no repositório do grupo (github.com/Igor-Novais/Startpoint).

5.1 Tecnologias utilizadas

Tabela 6 – Tecnologias do projeto

Parte	Tecnologias	Uso no projeto
Front-end	HTML5, CSS3 e JavaScript	Telas, layout responsivo (Flexbox, Grid e media queries), classes, fetch e localStorage.
Back-end	Node.js e Express	API REST com as rotas de usuários, cursos, trilha e vagas.
Bibliotecas do back-end	mysql2, bcryptjs, jsonwebtoken, dotenv e cors	Conexão com o banco, hash da senha, token de login, configurações no arquivo .env e liberação de acesso para o front-end.
Banco de dados	MySQL 8 e MySQL Workbench	Tabelas, chaves estrangeiras, índices, dados de exemplo e consultas.
Apoio	Figma Make, Trello e GitHub	Protótipo, quadro Kanban e controle de versão.

Uma mudança em relação ao plano da 2ª entrega: o plano citava o Bootstrap 5, mas o grupo preferiu escrever o CSS próprio a partir do protótipo, usando os recursos de layout vistos nas aulas de CSS. O resultado ficou mais próximo do visual do Figma.

5.2 Arquitetura: front-end e back-end

O sistema segue a arquitetura cliente-servidor em camadas. O **front-end** roda no navegador e só cuida da interface. O **back-end** é uma API REST que recebe as requisições, aplica as regras de negócio e consulta o banco. Os dois trocam dados em JSON. Depois do login, o front-end guarda o token JWT e o envia no cabeçalho `Authorization` de cada requisição.

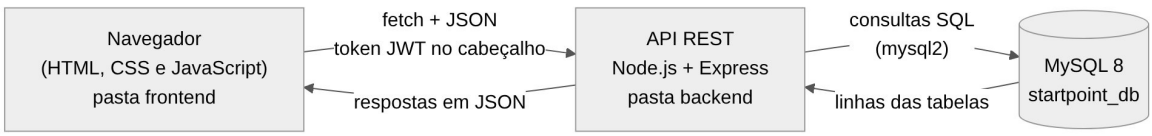


Figura 7 – Comunicação entre front-end, API e banco de dados.

Dentro do back-end, o código é separado no padrão MVC: as **rotas** definem os endereços da API, os **controllers** validam os dados e montam as respostas, e os **models** são classes que fazem as consultas SQL. O papel da "view" fica com as telas do front-end. O mesmo servidor entrega as páginas da pasta frontend, então basta abrir http://localhost:3000 no navegador.

5.3 Estrutura do projeto

Estrutura de pastas do repositório

```
Startpoint/
├── backend/
│   ├── src/
│   │   ├── config/db.js           conexão com o MySQL
│   │   ├── controllers/           regras de cada rota
│   │   ├── middlewares/auth.js    verificação do token JWT
│   │   ├── models/                classes com as consultas SQL
│   │   ├── routes/                endereços da API
│   │   └── server.js              sobe o servidor e entrega as telas
│   ├── testes/teste-api.js        teste automático da API
│   ├── .env.example               modelo das configurações
│   └── package.json
├── database/
│   ├── 01_criacao_tabelas.mwb     modelo no MySQL Workbench
│   ├── 02_Startpoint_SQL.sql      criação das tabelas e índices
│   ├── 03_dados_exemplo.sql       instituições, cursos, vagas e usuário de teste
│   └── 04_consultas.sql           consultas e comandos de teste
├── frontend/
│   ├── *.html                     12 telas
│   └── assets/css e assets/js     estilos e scripts das telas e os compartilhados
├── documentos/                   documentos das entregas
├── README.md
└── .gitignore                    não envia node_modules nem o .env
```

5.4 Banco de dados

O banco **startpoint_db** tem **10 tabelas**, 48 atributos e 10 chaves estrangeiras, normalizadas até a 3ª forma normal. As tabelas curso_competencia e vaga_competencia ligam cursos e vagas às competências e são a base do motor da trilha e do cálculo de aderência.

Em relação ao modelo da 2ª entrega, as mudanças foram:

- nova tabela **usuario_competencia**, que guarda o que o jovem marcou na etapa 2 do diagnóstico (antes isso não tinha onde ser gravado);
- campos **area** e **link_candidatura** na tabela vaga, usados no filtro por área e no botão “Quero me candidatar”;
- **ON DELETE CASCADE** nas chaves de trilha e progresso, para que, ao apagar um usuário, a trilha dele também seja apagada;
- **índices** em curso(area), vaga(area) e vaga(cidade), as colunas usadas nos filtros;
- script **03_dados_exemplo.sql** com 5 instituições, 18 competências, 19 cursos, 11 vagas em São Paulo/SP e um usuário de teste;
- arquivo **04_consultas.sql** com 25 itens para testar no Workbench: as consultas que as telas usam, consultas com GROUP BY, HAVING, CASE, BETWEEN, LIMIT, EXISTS e LEFT JOIN, e exemplos de INSERT, UPDATE e DELETE.

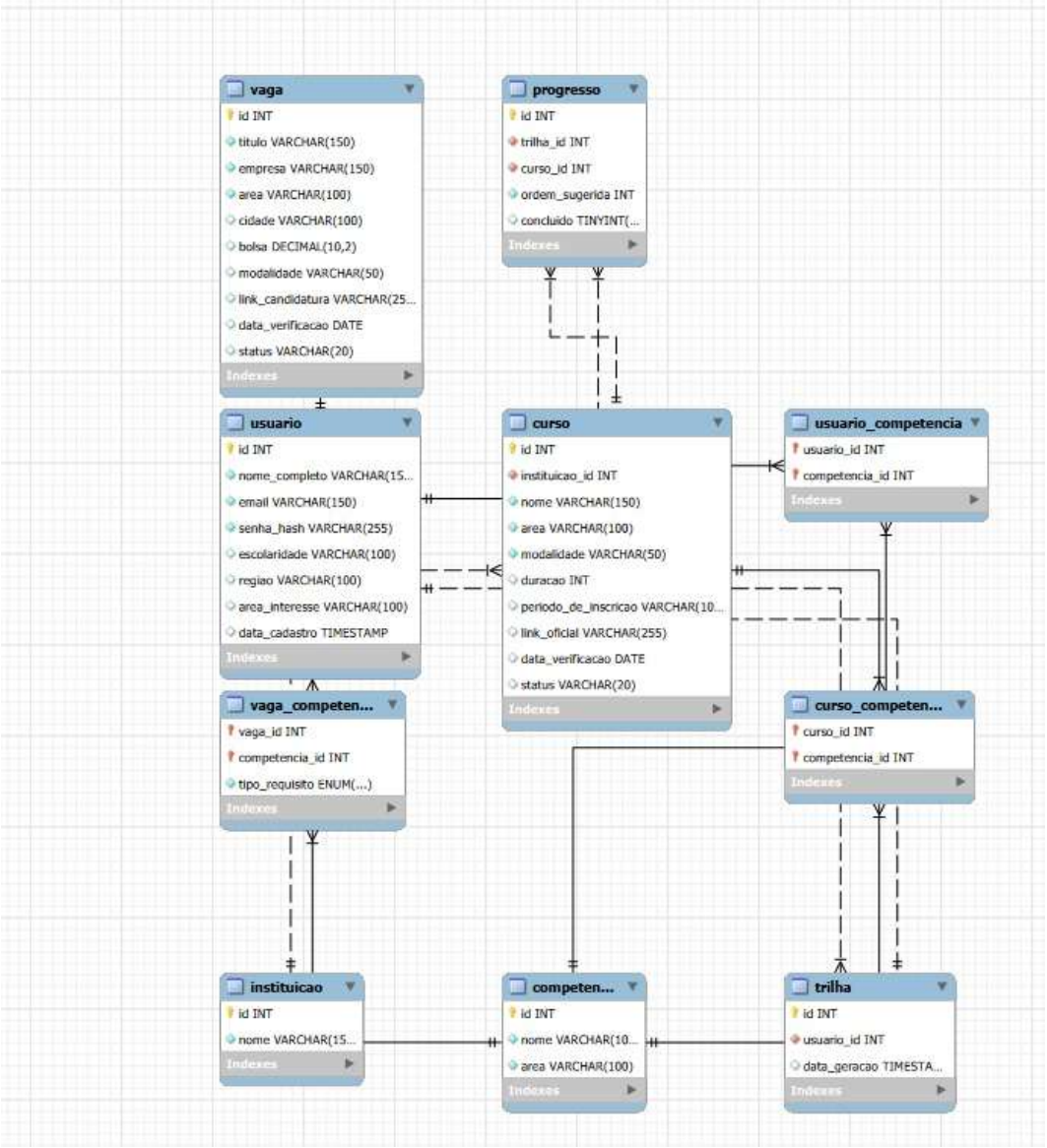


Figura 8 – Modelo do banco de dados com as 10 tabelas no MySQL Workbench (cada linha liga um registro a zero ou mais registros da outra tabela).

5.5 Telas implementadas

Foram implementadas **12 telas**: acesso, criar conta, recuperar senha, as quatro etapas do diagnóstico, Início, Minha trilha, Cursos, Vagas e Perfil. A estrutura e o visual seguem o protótipo e o HTML e CSS que o Conrado já tinha no repositório; os cards que antes eram fixos no HTML agora são montados pelo JavaScript com os dados da API. As figuras abaixo foram capturadas com o sistema rodando e o usuário de teste.



Figura 9 – Diagnóstico: etapa 2 (pode marcar mais de uma opção) e etapa 4 (área de interesse).

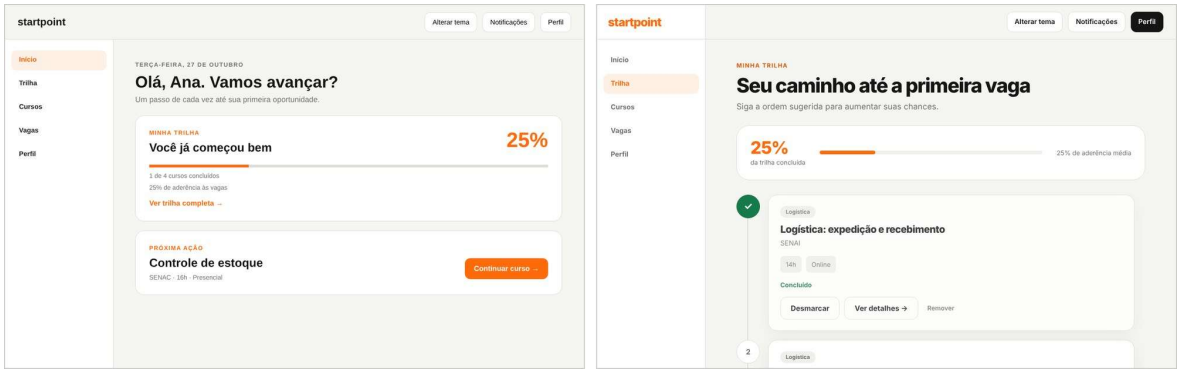


Figura 10 – Início e Minha trilha com um curso já concluído.

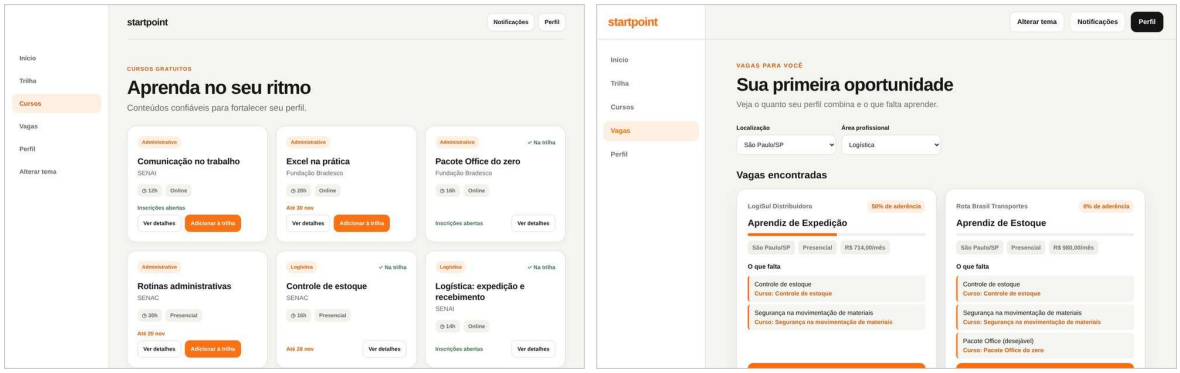


Figura 11 – Catálogo de cursos e mural de vagas com aderência e “O que falta”.

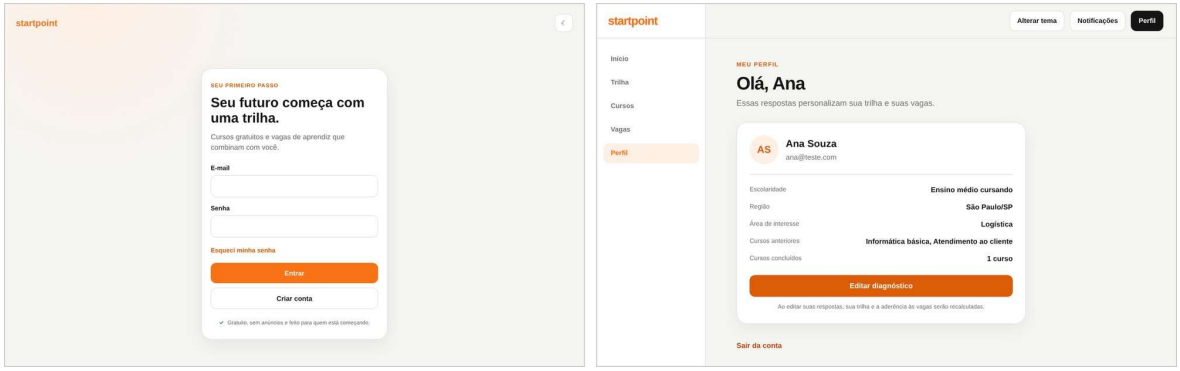


Figura 12 – Tela de acesso e Perfil com as respostas do diagnóstico.

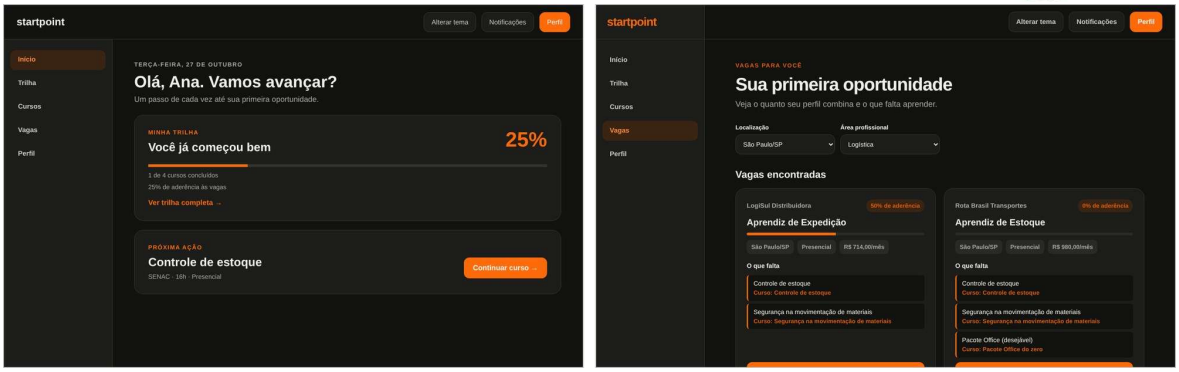


Figura 13 – Tema escuro, guardado no navegador para as próximas visitas.

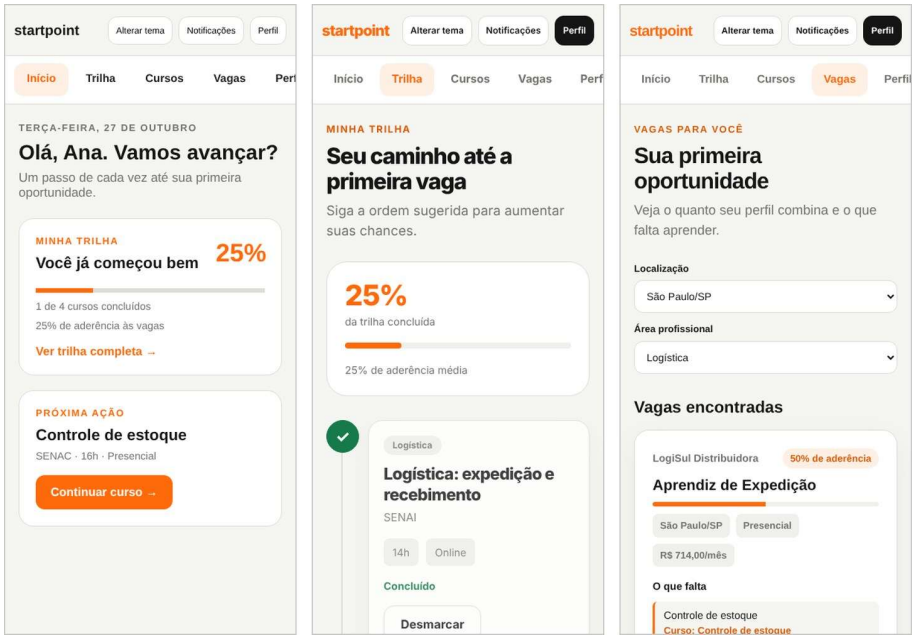


Figura 14 – Início, Minha trilha e Vagas no celular (390 px de largura).

5.6 Funcionalidades desenvolvidas e integração

A API tem 13 rotas. Com exceção do cadastro e do login, todas exigem o token no cabeçalho da requisição.

Tabela 7 – Rotas da API

Método	Rota	O que faz
POST	/api/usuarios	Cria a conta (senha com hash bcrypt)
POST	/api/usuarios/login	Confere e-mail e senha e devolve o token JWT
GET	/api/usuarios/perfil	Dados do usuário e do diagnóstico
PUT	/api/usuarios/diagnostico	Salva o diagnóstico
POST	/api/trilha	Gera (ou recalcula) a trilha
GET	/api/trilha	Trilha com percentual, aderência média e próximo curso
PATCH	/api/trilha/progresso/:id	Marca ou desmarca um curso como concluído
POST	/api/trilha/cursos	Adiciona um curso do catálogo na trilha

Método	Rota	O que faz
DELETE	/api/trilha/cursos/:id	Remove um curso da trilha
GET	/api/cursos	Catálogo de cursos ativos
GET	/api/cursos/areas	Áreas que têm curso (filtro da tela Vagas)
GET	/api/vagas	Vagas com aderência e “O que falta” (filtros: área e cidade)
GET	/api/vagas/cidades	Cidades que têm vaga

Login e sessão. O cadastro guarda só o hash da senha. No login, a API compara a senha com o hash e gera um token JWT válido por 8 horas. O front-end guarda o token no localStorage (como visto na aula de JavaScript Storage) e a classe `Api` o envia em todas as requisições. Do lado do servidor, um middleware confere o token antes de liberar as rotas protegidas:

Trecho de backend/src/middlewares/auth.js

```
function verificarToken(req, res, next) {
    const cabecalho = req.headers.authorization;

    if (!cabecalho || !cabecalho.startsWith('Bearer ')) {
        return res.status(401).json({ erro: 'Faça login para continuar.' });
    }

    const token = cabecalho.split(' ')[1];

    try {
        const dados = jwt.verify(token, process.env.JWT_SEGREDO);
        req.usuarioId = dados.id;
        next();
    } catch (erro) {
        return res.status(401).json({ erro: 'Sessão expirada. Entre novamente.' });
    }
}
```

Motor da trilha. Ao terminar o diagnóstico, o front-end salva as respostas e pede a geração da trilha. O model `Trilha` faz tudo dentro de uma **transação** (beginTransaction, commit e rollback, como na aula de transações de BDR), para que a trilha nunca fique pela metade se der erro. Os passos são: (1) cria a trilha do usuário ou reaproveita a existente; (2) mantém os cursos já concluídos; (3) busca as competências pedidas pelas vagas da área que o jovem ainda não tem; (4) busca os cursos que ensinam essas competências, já ordenados pelo banco (o CASE coloca primeiro os cursos da área do jovem); (5) percorre os cursos nessa ordem e só inclui os que ensinam algo novo; (6) se a área não tiver vagas, usa os cursos da própria área; (7) grava os cursos na ordem sugerida. A consulta do passo 4 é esta:

Trecho de backend/src/models/Trilha.js

```
SELECT c.id
FROM curso c
INNER JOIN curso_competencia cc ON cc.curso_id = c.id
WHERE c.status = 'ativo'
  AND cc.competencia_id IN (?)
  AND c.id NOT IN (SELECT curso_id FROM progresso WHERE trilha_id = ?)
GROUP BY c.id, c.area, c.duracao
ORDER BY CASE WHEN c.area = ? THEN 0 ELSE 1 END, COUNT(*) DESC, c.duracao
```

Aderência e "O que falta". A aderência de cada vaga é calculada no próprio SQL: o COUNT conta os requisitos da vaga e o SUM conta quantos deles estão na lista de competências do jovem. Para cada requisito que falta, uma subconsulta busca um curso que o ensina, de preferência da mesma área da vaga. Quando o jovem conclui um curso, as competências desse curso passam a contar e a aderência sobe na hora. Os filtros de área e de cidade só entram na consulta quando são escolhidos na tela:

Trecho de backend/src/models/Vaga.js

```
let sql = `
    SELECT v.id, v.titulo, v.empresa, v.area, v.cidade, v.bolsa, v.modalidade,
           v.link_candidatura,
           COUNT(vc.competencia_id) AS total_requisitos,
           SUM(CASE WHEN vc.competencia_id IN (?) THEN 1 ELSE 0 END)
               AS requisitos_atendidos
    FROM vaga v
    INNER JOIN vaga_competencia vc ON vc.vaga_id = v.id
    WHERE v.status = 'ativo'`;
const parametros = [tem];

if (filtros.area) {
    sql += ' AND v.area = ?';
    parametros.push(filtros.area);
}

if (filtros.cidade) {
    sql += ' AND v.cidade = ?';
    parametros.push(filtros.cidade);
}

sql += ' GROUP BY v.id';
sql += ' ORDER BY requisitos_atendidos / total_requisitos DESC, v.bolsa DESC';
```

Front-end com orientação a objetos. Os dados da API viram objetos de classes no arquivo modelos.js. A classe `ItemTrilha` herda de `Curso` e sobrescreve o método `situacao()` para mostrar "Concluído", aplicando herança e sobrescrita de métodos. Cada tela tem seu script (login.js, trilha.js, vagas.js etc.), que usa `addEventListener` para os eventos e `createElement` e `textContent` para montar os cards, como nas aulas de DOM e eventos.

5.7 Evolução desde a 2ª entrega

Tabela 8 – O que mudou da 2ª para a 3ª entrega

Parte	Na 2ª entrega	Na 3ª entrega
Telas	5 páginas em HTML, sem estilo	12 telas com CSS, responsivas e com tema escuro

Parte	Na 2ª entrega	Na 3ª entrega
Interatividade	Nenhuma; os dados estavam fixos no HTML	JavaScript em todas as telas, com dados vindos da API
Back-end	Só a estrutura de pastas	API REST com 13 rotas, login com JWT e motor da trilha
Banco	9 tabelas, sem dados	10 tabelas, índices, dados de exemplo e arquivo com 20 consultas e 5 comandos de teste
Integração	Não havia	Front-end, API e banco funcionando juntos
Testes	Não havia	18 testes automáticos da API e testes de interface (seção 6)

5.8 Situação atual

Já implementado: cadastro, login e sessão; diagnóstico e edição pelo Perfil; geração da trilha; catálogo de cursos; adicionar, remover e concluir cursos; mural de vagas com filtros de cidade e área, aderência, “O que falta” e atalho para o catálogo; confirmação de candidatura; tema escuro; layout para celular.

Em desenvolvimento: a base de cursos e vagas usa nomes reais de instituições, mas os cursos, as vagas e as empresas cadastrados são dados de exemplo criados pelo grupo para testar as regras. A troca por cursos e vagas reais, revisados com o parceiro da comunidade, está prevista para a Sprint 3.

Próximas etapas (Sprint 3): PWA, carga real de dados (incluindo as três áreas que ainda não têm cursos e vagas), envio de e-mail na recuperação de senha, testes com jovens do público-alvo e ajustes de layout apontados na seção 6.

6. Testes

Os testes tiveram dois objetivos: confirmar que cada funcionalidade faz o que os requisitos da seção 4 pedem e encontrar falhas antes da apresentação. A tabela abaixo resume os tipos de teste usados.

Tabela 9 – Tipos de teste realizados

Tipo	Para que serve	O que foi testado	Como
Integração	Ver se a API e o banco se comunicam corretamente	11 das 13 rotas da API, com o MySQL (as outras 2 só preenchem os filtros e foram testadas pela tela Vagas)	Script automático `npm test` (18 verificações)
Funcional	Ver se cada funcionalidade faz o que o requisito pede	Fluxo completo: cadastro, diagnóstico, trilha, cursos, vagas e perfil	Uso do sistema no navegador, seguindo os casos da seção 6.2
Validação de dados	Ver se entradas erradas são barradas com mensagem clara	Cadastro, login, diagnóstico e rotas sem token	Dados inválidos de propósito

Tipo	Para que serve	O que foi testado	Como
Interface	Ver telas, botões, navegação e responsividade	As 12 telas no computador (1280 px) e no celular (360 e 390 px), nos temas claro e escuro	Navegação manual e capturas de tela
Banco de dados	Ver se tabelas, chaves e consultas funcionam	Scripts de criação e de dados e as 20 consultas do 04_consultas.sql	Execução no MySQL
Usabilidade	Ver se o público entende e usa o sistema sem ajuda	Ainda não realizado com usuários reais	Previsto para a Sprint 3, com jovens indicados pelo parceiro

6.1 Teste automático da API (integração)

O arquivo backend/testes/teste-api.js cria um usuário novo, faz login, salva um diagnóstico, gera a trilha, conclui um curso e confere o resultado de cada passo, incluindo os casos de erro (e-mail repetido, senha errada e acesso sem token). Ele roda com o servidor ligado, pelo comando `npm test`. Na última execução, as 18 verificações passaram.



Figura 15 – Resultado do teste automático da API.

6.2 Casos de teste funcionais e de validação

Tabela 10 – Casos de teste

Caso	O que foi feito	Resultado esperado	Resultado
CT01	Criar conta com senhas diferentes	Mensagem “As senhas não são iguais.” e campo em destaque	OK
CT02	Criar conta com e-mail já cadastrado	Mensagem “Este e-mail já está cadastrado.”	OK
CT03	Entrar com senha errada	Mensagem “E-mail ou senha incorretos.”	OK
CT04	Abrir trilha.html sem estar logado	Voltar para a tela de login	OK
CT05	Chamar a API sem token	Resposta 401 “Faça login para continuar.”	OK
CT06	Clicar em Continuar no diagnóstico sem escolher nada	Mensagem “Escolha uma opção para continuar.”	OK
CT07	Diagnóstico com área Logística e Informática básica	Trilha com cursos de Logística primeiro	OK (4 cursos)
CT08	Marcar o primeiro curso como concluído	Percentual e aderência sobem	OK (0% para 25%; aderência de 13% para 25%)
CT09	Ver no catálogo um curso que já está na trilha	Selo “✓ Na trilha” e sem o botão Adicionar; pela API, repetir o curso devolve “Esse curso já está na sua trilha.”	OK
CT10	Trocar o filtro de área da tela Vagas para Tecnologia	Só vagas de Tecnologia	OK (2 vagas)
CT11	Clicar no atalho “Curso: ...” de uma vaga	Abrir o catálogo de cursos	OK
CT12	Clicar em “Quero me candidatar”	Confirmação com a quantidade de requisitos pendentes; se confirmar, abre o site da vaga em nova aba	OK
CT13	Editar o diagnóstico pelo Perfil	Respostas anteriores já marcadas e trilha recalculada	OK
CT14	Usar o tema escuro em todas as telas	Textos legíveis em todas as telas	Falha no Início, corrigida (seção 6.4)
CT15	Abrir as telas no celular (360 e 390 px)	Página sem rolagem lateral	OK; o menu rola para o lado para mostrar o Perfil, e a tela Cursos tem uma ressalva (seção 6.4)

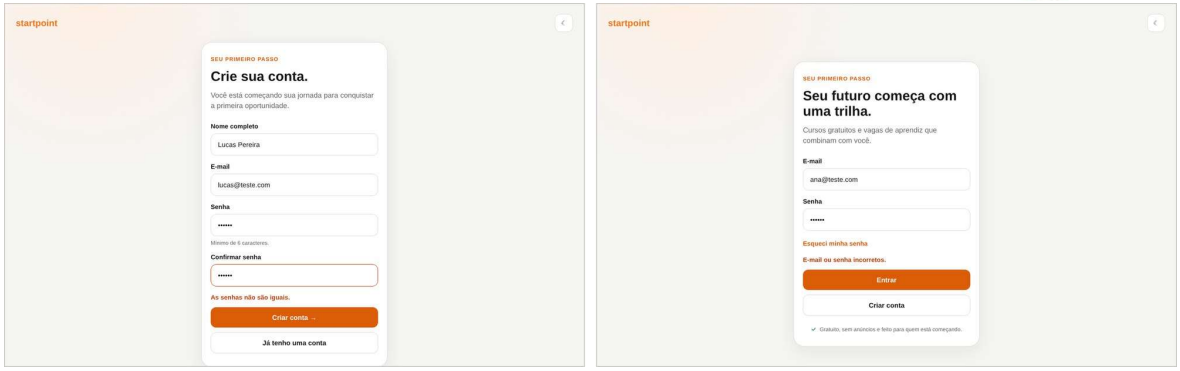


Figura 16 – CT01 e CT03: mensagens de validação no cadastro e no login.

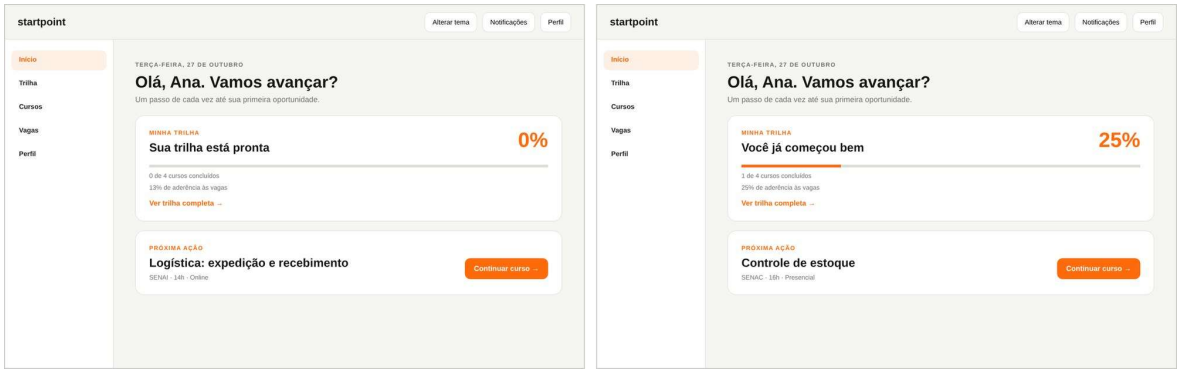


Figura 17 – CT08: Início antes e depois de concluir o primeiro curso da trilha.

6.3 Testes do banco de dados

Os scripts 02_Startpoint_SQL.sql e 03_dados_exemplo.sql foram executados do zero várias vezes, e as 20 consultas do 04_consultas.sql rodaram sem erro (os itens 21 a 25 são exemplos de INSERT, UPDATE e DELETE, deixados comentados). A figura abaixo mostra a consulta 6, que calcula a aderência da usuária de teste a cada vaga com as competências que ela marcou no diagnóstico (Informática básica e Atendimento ao cliente).

MySQL - startpoint_db

```
mysql> -- 6) Aderência do usuário a cada vaga (requisitos atendidos / total)
    -> SELECT v.titulo, v.empresa, v.bolsa,
    ->        COUNT(*) AS total_requisitos,
    ->        SUM(CASE WHEN vc.competencia_id IN (SELECT competencia_id FROM usuario_competencia
    ->                                            WHERE usuario_id = 1) THEN 1 ELSE 0 END) AS atendidos,
    ->        ROUND(SUM(CASE WHEN vc.competencia_id IN (SELECT competencia_id FROM usuario_competencia
    ->                                                  WHERE usuario_id = 1) THEN 1 ELSE 0 END) / COUNT(*) * 100) AS aderencia
    -> FROM vaga v
    -> INNER JOIN vaga_competencia vc ON vc.vaga_id = v.id
    -> WHERE v.status = 'ativo'
    -> GROUP BY v.id, v.titulo, v.empresa, v.bolsa
    -> ORDER BY aderencia DESC;
+-----------------------------------+----------------------------+---------+-----------------+-----------+-----------+
| titulo                            | empresa                    | bolsa   | total_requisitos | atendidos | aderencia |
+-----------------------------------+----------------------------+---------+-----------------+-----------+-----------+
| Aprendiz Operador de Caixa        | Supermercado Bom Preço     |  780.00 |               3 |         2 |        67 |
| Aprendiz de Suporte de TI         | Conecta TI Serviços        | 1100.00 |               4 |         2 |        50 |
| Aprendiz em Desenvolvimento Web   | Vértice Sistemas           | 1200.00 |               3 |         1 |        33 |
| Aprendiz de Recepção              | Centro Empresarial Paulista |  850.00 |               3 |         1 |        33 |
| Aprendiz de Vendas                | Loja Ponto Certo           |  820.00 |               3 |         1 |        33 |
| Aprendiz de Expedição             | LogiSul Distribuidora      |  714.00 |               4 |         1 |        25 |
| Aprendiz de Recepção em Clínica   | Clínica Bem Viver          |  900.00 |               4 |         1 |        25 |
| Aprendiz de Estoque               | Rota Brasil Transportes    |  980.00 |               3 |         0 |         0 |
| Aprendiz Auxiliar Administrativo  | Andrade Contabilidade      |  900.00 |               4 |         0 |         0 |
| Aprendiz de Apoio Hospitalar      | Hospital Santa Clara       |  950.00 |               3 |         0 |         0 |
+-----------------------------------+----------------------------+---------+-----------------+-----------+-----------+
```

Figura 18 – Consulta 6 do 04_consultas.sql executada no MySQL.

6.4 Falhas encontradas e correções

- **Caminhos quebrados:** as páginas procuravam os arquivos CSS fora da pasta assets/css, e o menu levava a curso.html, que não existe. Os caminhos foram corrigidos em todas as telas.
- **Formulários recarregando a página:** o formulário de login tinha action="/login" e os outros action="#", então todos enviavam a página inteira. Passaram a usar JavaScript com preventDefault e fetch para falar com a API.
- **Respostas do diagnóstico:** as opções usavam códigos (como medio_cursando) que não batiam com os nomes gravados no banco. Os valores passaram a ser o próprio texto da opção.
- **Trilha com cursos fora da área:** na primeira versão do motor, um usuário de Tecnologia recebia, por exemplo, o curso Vendas e negociação. A regra passou a priorizar os cursos da área e a incluir só cursos que ensinam alguma competência nova.
- **Erro de SQL com lista vazia:** para um usuário sem nenhuma competência, a consulta com IN () dava erro. O código passou a tratar a lista vazia.
- **Acentos ao importar pelo terminal:** ao rodar os scripts pelo terminal do MySQL, os acentos ficavam errados. No Workbench isso não acontece; pelo terminal, é preciso usar a opção --default-character-set=utf8mb4.
- **Tema escuro no Início:** os botões do topo ficavam com texto preto sobre fundo escuro. Foi corrigido no comum.css.
- **Pendente:** na tela Cursos, no celular, o menu continua na lateral e ocupa parte da tela; e o botão Notificações, que aparece no topo de todas as telas, ainda não tem função. Os dois ajustes ficaram para a Sprint 3.

7. Implantação

Nesta fase, o Startpoint roda em **servidor local**, que é a forma usada na apresentação. O mesmo servidor Node.js entrega a API e as telas.

7.1 Ambiente necessário

- Node.js (versão LTS) com o npm;
- MySQL 8 e MySQL Workbench;
- navegador atualizado (Chrome, Edge ou Firefox).

7.2 Como executar

1. No MySQL Workbench, abrir e executar database/02_Startpoint_SQL.sql e depois database/03_dados_exemplo.sql.
2. Na pasta backend, copiar o arquivo .env.example para .env e colocar a senha do MySQL.
3. Na pasta backend, rodar `npm install` (só na primeira vez) e depois `npm start`.
4. Abrir http://localhost:3000 no navegador e entrar com o usuário de teste (ana@teste.com, senha 123456) ou criar uma conta.

Terminal - pasta backend

```
startpoint/backend$ npm start

> startpoint-backend@1.0.0 start
> node src/server.js

Servidor rodando em http://localhost:3000
Conectado ao MySQL (startpoint_db)
```

Figura 19 – Servidor iniciado e conectado ao banco.

O arquivo .env, que tem a senha do banco, não vai para o GitHub (está no .gitignore). Cada integrante cria o seu a partir do .env.example.

7.3 Requisitos mínimos para uso

Para o jovem, basta um navegador atualizado no celular ou no computador e conexão com a internet; não é preciso instalar nada. As telas foram testadas com 360 px e 390 px de largura.

7.4 Publicação prevista

Para publicar na internet, as telas podem ir para um serviço de hospedagem de sites estáticos, mas a API precisa de um serviço que rode Node.js e de um banco MySQL na nuvem. Nesse caso, basta mudar o endereço da API no arquivo frontend/assets/js/api.js e as variáveis do .env no servidor. Com o PWA da Sprint 3, o jovem também poderá adicionar o Startpoint à tela inicial do celular.

8. Manutenção / Evoluções Futuras

8.1 Limitações atuais do MVP

- Cursos, vagas e empresas cadastrados ainda são dados de exemplo.
- O sistema atende só a cidade de São Paulo/SP, e três das oito áreas do diagnóstico ainda não têm dados.
- A recuperação de senha mostra a mensagem, mas não envia e-mail.
- O checklist de contratação, a busca por texto nas telas Cursos e Vagas e os filtros da tela Cursos ficaram fora do MVP por causa do prazo.
- O botão Notificações ainda não tem função.
- O sistema roda localmente e ainda não foi testado por jovens do público-alvo.

8.2 Melhorias e evoluções previstas

- **PWA:** manifest.json e service worker para instalar o Startpoint no celular e guardar as telas já visitadas (Sprint 3).
- **Base real:** cadastrar cursos e vagas reais, revisados com o parceiro da comunidade, e completar as áreas Atendimento ao Cliente, Serviços Financeiros e Indústria.
- **Curadoria mais fácil:** um painel administrativo para cadastrar e revisar cursos e vagas sem escrever SQL, com aviso dos registros sem verificação há mais de 15 dias (a consulta 11 do 04_consultas.sql já faz isso para os cursos).
- **Checklist de contratação:** a tela do protótipo, com os documentos para a contratação e dicas de currículo e entrevista, salva na conta do usuário.
- **Busca:** campo para buscar cursos por nome ou instituição e vagas por empresa ou cargo.
- **Notificações:** avisar quando abrir inscrição de um curso da trilha ou quando surgir uma vaga com aderência alta.
- **Ampliação do público:** outras cidades da região metropolitana e novas áreas profissionais.
- **Segurança:** envio de e-mail para recuperar a senha, limite de tentativas de login e HTTPS na publicação.
- **Interface:** menu da tela Cursos no topo também no celular, como nas outras telas.
- **Publicação:** colocar a API e o banco em um serviço na nuvem para o sistema ser acessado de qualquer lugar.

9. Considerações Finais

O Startpoint partiu de um problema concreto: o jovem da periferia tem cursos gratuitos à disposição, mas não sabe quais existem, o que as vagas pedem nem por onde começar. A proposta foi organizar essa informação em um caminho único para cada jovem, ligando o que ele já sabe ao que as vagas de aprendiz exigem.

Nesta 3ª entrega, a proposta deixou de ser só protótipo. O sistema funciona de ponta a ponta: o jovem cria a conta, responde ao diagnóstico, recebe uma trilha calculada a partir das vagas da área, acompanha o próprio progresso e vê a aderência às vagas subir conforme conclui os cursos. Front-end, API e banco de dados estão integrados e foram testados, e as falhas encontradas foram corrigidas ou registradas para a próxima sprint.

Para o grupo, o projeto colocou em prática, num mesmo sistema, o que foi visto em Programação Web (HTML, CSS, JavaScript, armazenamento no navegador e JWT) e em Bancos de Dados Relacionais (modelagem, normalização, índices, consultas, orientação a objetos e transações), além da organização com Scrum e Kanban. A etapa final será a carga de dados reais com o parceiro, o PWA e os testes com o público, para que o Startpoint possa de fato ajudar jovens a chegar à primeira vaga.